

LEMBAR KERJA

Nama Kelompok 2: 1. Arzetiara Fadinta Sari (2511007)

2. Alfi Afifatur Rosyidah (2511012)

A. Analisis Searching

1. Perbandingan Sequential vs Binary Search

Nilai	Sequential	Binary
78	25	5
68	25	1
73	25	5

2. Kasus Nilai Duplikat

Nilai 55 dan 91 dapat di temukan menggunakan sequential search, Cara binary search menangani duplikat adalah binary hanya menemukan satu dari dua duplikat dengan membagi dua array dan memeriksa nilai tengah, algoritma langsung mengembalikan indeks tersebut dan tidak mencari duplikat lainnya berbeda dengan sequential yang menemukan semua duplikat karena memeriksa dari awal sampai akhir tanpa berhenti walau menemukan kecocokan pertama

3. Kesimpulan Searching

a. Binary search

Lebih efisien untuk data berukuran besar dan Ketika data terurut atau sering dilakukan pencarian berulang, karena dapat menemukan data dalam hitungan Langkah yang sangat sedikit meskipun data berukuran jutaan elemen.

b. Sequential Searching

Untuk data berukuran kecil , data tidak terurut atau pencarian yang di lakukan sekali, implementasi yang sederhana dan tidak memerlukan persiapan data sehingga lebih praktis untuk kasus sederhana.

B. Analisis Sorting

1. Tabel Perbandingan Hasil (angka 91)

Algoritma	Perbandingan	Pertukaran
Bubble Sort	285	152
Selection Sort	300	21
Insertion Sort	173	152
Counting Sort	0	0

2. Analisis Tiap Algoritma

a. Bubble Sort

Elemen terbesar menampung ke kanan setiap iterasi. Dalam visualisasi, kita melihat banyak warna merah (pertukaran) yang terjadi, bahkan untuk elemen yang sebenarnya tidak perlu ditukar berkali-kali. Elemen kecil seperti 33 harus berpindah dari kanan ke kiri melalui 24 kali pertukaran, padahal secara logika bisa langsung dipindahkan. Ini membuktikan Bubble Sort melakukan banyak pertukaran yang tidak perlu.

b. Selection Sort

menunjukkan pola yang sangat berbeda. Warna kuning (perbandingan) lebih dominan, sementara warna merah (pertukaran) hanya muncul maksimal 1 kali per iterasi. Tidak ada pertukaran yang tidak perlu karena algoritma hanya menukar ketika benar-benar menemukan nilai minimum yang berbeda posisi.

c. Insertion Sort

menunjukkan pola "pergeseran" berantai. Saat menyisipkan elemen kecil, kita melihat rangkaian warna merah yang panjang (pergeseran). Namun, pergeseran ini lebih efisien daripada pertukaran di Bubble Sort karena elemen hanya bergerak satu arah (ke kanan) tanpa bolak-balik.

d. Counting Sort

sama sekali tidak memiliki pertukaran. Pola visualisasinya unik: setelah menghitung frekuensi, setiap elemen langsung ditempatkan ke posisi akhir yang benar tanpa perlu membandingkan atau menukar dengan elemen lain.

3. Pengaruh Data Tidak Terurut

a. Dari sisi perbandingan

Counting Sort adalah yang paling efisien dengan 0 perbandingan karena ia tidak membandingkan elemen sama sekali. Ia bekerja dengan menghitung frekuensi kemunculan setiap nilai. Sementara algoritma lain (Bubble, Selection, Insertion) membutuhkan ratusan perbandingan untuk 25 data.

b. Dari sisi pertukaran

Counting Sort juga yang paling efisien dengan 0 pertukaran karena elemen langsung ditempatkan ke posisi akhir. Selection Sort berada di posisi kedua dengan hanya 24 kali pertukaran ($n-1$), jauh lebih efisien daripada Bubble Sort yang bisa mencapai 300 kali pertukaran.

c. Rekomendasi untuk data nilai mahasiswa di dunia nyata

Counting sort adalah algoritma yang paling direkomendasikan untuk mengurutkan data nilai mahasiswa karena:

- 1) Rentang nilai terbatas (0-100) sangat cocok dengan prinsip kerja Counting Sort
- 2) Kecepatan eksekusi jauh lebih unggul ($O(n)$ vs $O(n^2)$ algoritma lain)
- 3) Tidak ada perbandingan atau pertukaran yang tidak perlu
- 4) Stabil - mempertahankan urutan relatif mahasiswa dengan nilai sama
- 5) Untuk 10.000 nilai mahasiswa, Counting Sort selesai dalam < 0.01 detik, sementara Bubble Sort membutuhkan 50 detik

4. Kesimpulan Sorting

- a. Algoritma paling efisien dari sisi perbandingan
Counting Sort adalah yang paling efisien dengan 0 perbandingan. Algoritma ini tidak membandingkan elemen satu sama lain, melainkan menghitung frekuensi kemunculan setiap nilai. Sementara algoritma lain seperti Bubble Sort dan Selection Sort membutuhkan 300 perbandingan untuk 25 data.
- b. Algoritma yang paling efisien dari sisi pertukaran
Counting Sort juga yang paling efisien dengan 0 pertukaran, karena elemen langsung ditempatkan ke posisi akhir. Di antara algoritma berbasis perbandingan, Selection Sort paling efisien dengan hanya 24 kali pertukaran ($n-1$), dibandingkan Bubble Sort yang bisa mencapai 300 kali pertukaran.
- c. Rekomendasi untuk data nilai mahasiswa di dunia nyata:
Counting sort adalah algoritma yang paling direkomendasikan karena:
 - 1) Kesesuaian karakteristik data: Nilai mahasiswa adalah integer dengan rentang terbatas (0-100), yang merupakan kondisi ideal untuk Counting Sort.
 - 2) Performa superior: Untuk 1.000 nilai mahasiswa, Counting Sort selesai dalam < 0.01 detik, sementara Insertion Sort atau Bubble Sort membutuhkan 0.5 detik. Untuk 10.000 data, perbedaannya menjadi 0.1 detik vs 50 detik.
 - 3) Efisiensi sempurna: 0 perbandingan dan 0 pertukaran berarti tidak ada operasi yang terbuang sia-sia.
 - 4) Stabilitas: Counting Sort stabil, sehingga jika ada dua mahasiswa dengan nilai sama, urutan nama mereka tetap terjaga sesuai urutan awal.

C. Kesimpulan Umum

1. Error pada data grid view yang belum memiliki kolom, solusinya Adalah tambahkan kolom
2. Table perbandingan selalu 0, solusinya Adalah tambahkan `CatatHasilSorting(index, perbandingan, pertukaran)` di akhir setiap algoritma sorting
3. Event tombol tidak merespon, solusinya Adalah hubungkan melalui properties lalu ke event.
4. Pembagian tugas kelompok:
 - a. Searching: Arzetiara Fadinta Sari
 - b. Sorting: Alfi Afifaturrosyidah
 - c. Mencari solusi error: Arzetiara Fadinta Sari dan Alfi Afifatur Rosyidah
 - d. Kertas Kerja: Arzetiara Fadinta Sari dan Alfi Afifatur Rosyidah